Advanced Topics & Independent Research

Kyllian LIM & Ömer ÖZENLER

1. A* Pathfinding:

Why we chose the topic and how it is relevant to our project:

In our tower defense, enemies must find their way from their spawn cells on the left side of the field to the goal on the right while avoiding the towers. A* is the standard solution to this problem and is used in many video games, so it was a natural fit. Furthermore, one of us already knew the algorithm and enjoys graph theory, so we did not start from nothing. We also simply found it a fun algorithm.

What we learned through our research:

A* expands the node with the lowest f(n) = g(n) + h(n), where g(n) is the best-known cost from the start and h(n) estimates the remaining cost to the goal. It can be seen as Dijkstra's algorithm guided by a heuristic, allowing it to explore fewer nodes. If the heuristic never overestimates the real cost (it is admissible), A* still guarantees a shortest path. The algorithm also uses an open set (priority queue), a closed set of expanded nodes, the best-known g-values, and a "came from" map to reconstruct the final path.

How we applied the topic in our project:

In our Astar class, the field is a 9x11 grid of Node objects. Each node knows its four neighbours and whether a tower stands on it. findPath(start) stores g, the parents and the closed set in two HashMaps and a HashSet. It uses our own priority queue, ordered by f, as the open set. Once the goal is dequeued, reconstructPath follows the parents back to the start and reverses the list. Furthermore, in EnemyManager, each enemy stores its current node, asks A* for a path from there and moves pixel by pixel towards the next node. Additionally, we first used the Euclidean distance as the heuristic but switched to the Manhattan distance. Indeed, the Manhattan distance is equal to the real path cost when no towers are in the way. Because enemies only move in four directions, this makes it a great estimation.

What we learned from the implementation:

Our first version contained several mistakes. We accidentally added the heuristic to the cost from the start. A node's f value became its neighbours' past cost, so heuristics piled up along the path and paths were no longer compared correctly. Storing g separately and using f only as the queue priority fixed this. Our parent logic was also wrong. Indeed, we rebuilt the path by scanning a list of (node, parent) pairs for matching IDs. A parent map made this simpler

and more reliable. However, one limitation remains. Indeed, findPath runs for every enemy on every 16 ms tick, even though paths only change when a tower is placed.

2. Priority Queue:

Why we chose the topic and how it is relevant to our project:

A* repeatedly needs to remove the node with the lowest f value from its open set, which is exactly what a priority queue does. Java already offers java.util.PriorityQueue, but we wanted to challenge ourselves and understand the data structure from the inside instead of going for the easy option. One of us already knew what a queue is and how to implement one as a linked list, so we could build on that and adapt it into a priority queue.

What we learned through our research:

A priority queue is an abstract data type where elements are inserted together with a priority, and the element with the best priority is always removed first. A sorted linked list inserts in O(n), because it must walk to the right position, but removes the minimum in O(1) by taking the head. A binary heap, which is what java.util.PriorityQueue uses, does both operations in O(log n).

How we applied the topic in our project:

Our PriorityQueue is a linked list of Link objects, each holding a node, its priority as a double and a reference to the next link. It is sorted from lowest to highest priority. enqueue walks the list and inserts the new link after all links with an equal or lower priority and therefore keep their insertion order. dequeue removes the head.

What we learned from the implementation:

Making the queue work with A* was the main difficulty, as we had to adapt its data structure and variable types. Initially, the priority was an attribute of the node, and every insertion required re-sorting the entire queue with insertion sort, costing $O(n^2)$. Moreover, enqueue called the sort, which called enqueue again, causing infinite recursion. We therefore learned to keep the queue sorted during insertion and separate the priority from the element. Although Java's heap-based queue is more efficient (O(log n) instead of O(n)), the difference is negligible for our grid of 99 cells. Finally, we replaced NullPointerException with IllegalStateException when dequeuing an empty queue, as it better describes the error. A* always checks isEmpty() first, so this exception should only occur if there is a bug.